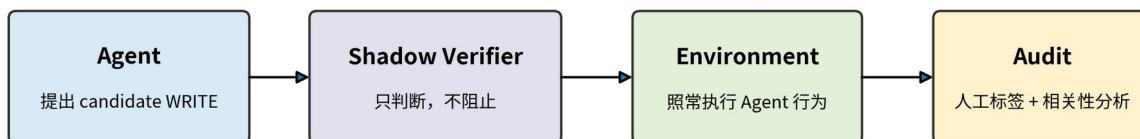


Agent Systems 研究型进阶专题 A · 第 5 讲

两个会犯错的 AI 组件组合后，系统为什么可能仍然不可靠

本讲定位：第 4 讲已经有了 Pre-Write Verifier。第 5 讲不再问“有没有提升”，而是问：Agent 与 Verifier 是否在同一批困难样本上共同犯错？这一步把研究对象从单组件准确率推进到系统级错误结构。

核心：测量组件错误，不让 Verifier 改变 Agent 的行为分布



记录：candidate tool call / verifier decision / reason / cost

之后再人工判断 proposal 是否真的 VALID / INVALID

图 1 Shadow Verifier: 只观察、不干预，专门用于测量组件错误结构

1. 先回答一个反直觉问题：多一个 Verifier，为什么不一定更可靠？

假设 Agent 有 20% 的概率提出错误 WRITE，Verifier 自己只有 10% 的错误率。很多初学者会下意识认为：“那危险错误大概就是 $20\% \times 10\% = 2\%$ 。”这个结论只有在一个非常强的条件下才成立：两者的错误近似独立。

底层原理：系统风险取决于“错误是否一起发生”，而不只取决于每个组件单独有多准。

如果 Agent 和 Verifier 都被同一种歧义、同一段缺失信息、同一种 policy 表述或同一模型家族的盲点迷惑，那么 Agent 越容易错的地方，Verifier 也可能越容易放过。此时“10% Verifier error”并不能按独立假设理解。

“两个 90% 可靠的组件”仍可能因为错误相关而形成完全不同的系统风险

同样的边际错误率	Agent error = 20%	Verifier error = 10%	危险双错
负相关 / 互补	20 个 Agent 错误里 0 个被 Verifier 漏过	$\kappa = 0$	0%
近似独立	预计 $20\% \times 10\%$	$\kappa \approx 1$	2%
正相关 / 共盲点	Agent 错的地方 Verifier 也更容易错	$\kappa > 1$	例如 6%

$$\kappa = P(\text{Agent error} \cap \text{Verifier error}) / [P(\text{Agent error}) \times P(\text{Verifier error})]$$

图 2 边际错误率相同，但系统危险双错率可以完全不同

2. 本讲最重要的三个随机变量

变量	取值 1 的含义	怎么得到
A: Agent proposal error	Agent 提出的 WRITE 实际上不应执行	人工结合 policy + 已观察状态标注 INVALID
V: Verifier error	Verifier 的 ALLOW/RECONSIDER 与人工标签相反	由 verifier decision 与人工标签比较得到
D: Dangerous co-failure	A=1 且 V=1，即 Agent 错、Verifier 也漏过	INVALID + ALLOW

这里最危险的不是普通 Verifier error，而是 D：Agent 已经提出错误写操作，Verifier 又恰好认为可以放行。它才是系统层面的“危险双错”。

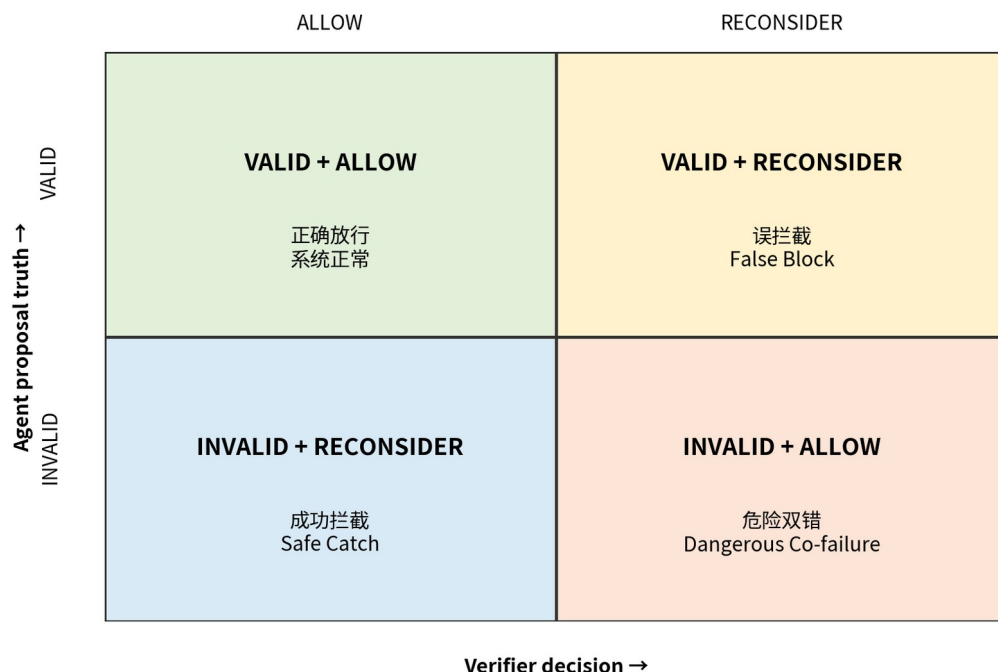


图3 Agent proposal truth × Verifier decision 的四象限

3. 为什么第 5 讲必须用 Shadow Mode ？

第 4 讲的 Pre-Write Verification 会真正阻止 WRITE。它适合验证“干预有没有用”，但不适合直接测量原始错误相关性，因为一旦 Verifier 阻止了某个动作，后续 trajectory 就已经改变。

Shadow Verifier 的做法是：Agent 正常提出动作；Verifier 在旁边判断，但无论它说 ALLOW 还是 RECONSIDER，系统都照 Agent 原来的动作执行。这样我们得到的是“同一份 Agent proposal 上，Verifier 会怎么判断”。

实验设计原则：先测量，再干预。测量错误结构时尽量不要改变被测系统；验证改进方案时再真正接管控制。

- 优点 1：不会因为拦截动作而改变后续数据分布。
- 优点 2：可以把 Agent proposal 与 Verifier decision 一一配对。
- 优点 3：方便比较“同模型 Verifier”和“异模型 Verifier”的共盲点。
- 代价：多一次 LLM 调用，因此真实部署时会增加费用和时延；但教学研究阶段先把测量做干净。

4. 人工标签为什么不能偷懒用 reference actions 代替？

τ-bench 的 reference actions 通常是一条可行轨迹，不是唯一正确轨迹。一个 candidate WRITE 没有与 reference action 完全匹配，并不自动等于错误。因此本讲脚本只把 reference exact match 当作诊断字段，字段名里直接写了 DIAGNOSTIC_ONLY。

真正的标签必须回答：在这个时刻，根据 policy、用户已经提供的信息和 tool 已经返回的状态，这个 WRITE 是否应该执行？

标签	含义	何时使用
VALID	当前动作可以执行，关键参数和前置条件都有依据	证据充分
INVALID	当前动作不应执行：违反 policy、缺前置条件、参数猜测等	可以明确指出错误
UNCERTAIN	你暂时无法可靠判断	证据不足或任务定义有歧义；统计时先排除

常见误区：“gold action 里没有这个工具调用，所以它错了。”——不成立。reference trajectory 不是唯一正确路径。

5. 两个最重要的相关性指标：κ 和 φ

5.1 先看最直观的 κ (co-failure ratio)

先计算危险双错的实际概率： $P(A \cap V)$ 。再计算如果两种错误彼此独立时应该出现多少： $P(A) \times P(V)$ 。

公式： $\kappa = P(A \cap V) / [P(A) \times P(V)]$		
κ	直观含义	科研解释
≈ 1	实际共错接近独立预期	暂时没有明显共盲点信号
> 1	共同失败比独立预期更多	正相关 / shared blind spot 候选
< 1	共同失败比独立预期更少	互补性 / error decorrelation 候选

5.2 φ (phi coefficient)

φ 是两个二元变量的相关系数。这里把 A=Agent 是否错、V=Verifier 是否错都编码成 0/1，就可以得到 -1 到 +1 的 φ。小样本时不要把它当成“定论”，主要用来做方向性诊断。

n_{11} = Agent 错 且 Verifier 错 n_{10} = Agent 错 且 Verifier 对 n_{01} = Agent 对 且 Verifier 错 n_{00} = Agent 对 且 Verifier 对 $\phi = \frac{(n_{11} \cdot n_{00} - n_{10} \cdot n_{01})}{\sqrt{(n_{11} + n_{10})(n_{01} + n_{00})(n_{11} + n_{01})(n_{10} + n_{00})}}$
--

6. 一个数字例子：为什么“各自都挺准”仍然可能危险

假设我们审计 100 个 WRITE proposals。Agent 错 20 个；Verifier 总共错 10 个。边际看起来都不算离谱。

情形	危险双错 n11	$P(A \cap V)$	独立预期	κ
互补	0	0%	2%	0
近似独立	2	2%	2%	1
共享盲点	6	6%	2%	3

这三个系统的 Agent error rate 都是 20%，Verifier error rate 都是 10%，但真正危险的双错可以从 0% 到 6%。所以系统科学真正关心的是 joint structure，而不是只看单组件 accuracy。

7. 错误为什么会正相关？

- 共同输入困难：同一任务缺信息、长上下文、policy 条件嵌套，Agent 和 Verifier 都在最难的样本上吃亏。
- 共同模型家族：生成者和监督者共享相似表示、训练数据或推理习惯，可能形成同源盲点。
- 共同证据缺失：Agent 没读到关键状态，Verifier 也只能看到同一份不完整上下文。
- 共同提示偏差：系统 prompt 对某个规则强调不足，两边都沿同一错误方向解释。
- 错误传播：上游 Agent 先产生一个看似合理的错误解释，Verifier 被这个解释锚定。

科研洞察：“更强的第二个模型”不自动等于“更可靠的监督”。真正有价值的设计往往是让监督组件拥有不同证据、不同模型、不同工具或不同决策结构，从而制造互补性。

8. 本讲真实实验：3 个任务 × 4 trials 的 Shadow Verifier Audit

为了控制成本，本讲先做 pilot：沿用第 3~4 讲已经精读过的 3 个 retail tasks，每个跑 4 次。真正的样本单位不是 episode，而是“WRITE proposal”。一个 episode 可能没有 WRITE，也可能出现多个 WRITE。

1. 运行 ShadowVerifyAgent：Agent 正常行动，Verifier 只记录 ALLOW / RECONSIDER。
2. 提取所有 WRITE proposals：candidate tool、参数、上下文、Verifier 理由。
3. 人工标注每条 proposal 为 VALID / INVALID / UNCERTAIN。
4. 运行 analyze_error_correlation.py，得到四象限、 κ 、 ϕ 、task-level 表。
5. 优先精读 INVALID + ALLOW 的 dangerous_cases.csv。
6. 把重复出现的共同盲点写成 Hypothesis Card，再设计下一轮最小干预。

从“Verifier 有没有用”升级为“它在哪些错误上和 Agent 共盲？”



这一步产生的不是最终论文结论，而是可检验的系统机制假设

图4 第5讲实验闭环：从 Shadow Trace 到可证伪共盲点假设

9. 保姆式命令流程

```
# 0. 在 tau2-bench 项目根目录执行
python /path/to/a05/src/00_tau2_compat_check.py

# 1. 跑 Shadow Verifier (示例 task id 请替换成你第3-4讲实际选的)
python /path/to/a05/src/run_shadow_audit.py --task-ids 17 41 23 --agent-llm <你的 Agent 模型> --verifier-llm <你的 Verifier 模型> --user-llm <用户模拟器模型> --num-trials 4 --save-to a05_shadow_audit

# 2. 提取待人工标注的 WRITE proposals
python /path/to/a05/src/prepare_write_annotation.py data/simulations/a05_shadow_audit/results.json --out a05_write_annotation.csv

# 3. 手工填 VALID / INVALID / UNCERTAIN 后分析
python /path/to/a05/src/analyze_error_correlation.py a05_write_annotation.csv --out a05_analysis
```

10. 结果怎么解释：不要把 $\kappa > 1$ 直接写成“证明了共享机制”

结果	可以说什么	不能直接说什么
$\kappa > 1$ 、 $\phi > 0$ ，且危险双错集中于同类任务	存在正向共错信号，值得精读共盲点	已经证明某个因果机制
$\kappa \approx 1$	当前样本没有明显超出独立预期的共错	组件一定独立
$\kappa < 1$	可能存在互补性/错误去相关	这个组合已经“安全”
不同 verifier model 的 κ 不同	模型异质性可能影响监督互补性	异模型一定优于同模型

标准科研流程：观察统计信号 → 精读 Trace → 提出机制假设 → 设计只改变一个因素的实验 → 再验证。不

要从一个相关系数直接跳到结论。

11. 与当前研究的关系：这不是我们凭空发明的问题

已有研究已经明确指出“模型相似性与共错会削弱 AI oversight”，也有 2026 年工作专门审计 LLM judge 在 agentic tool-calling、格式扰动和复杂轨迹上的可靠性。本讲的目标不是声称“错误相关性”本身是新问题，而是学会在自己的 Agent System 中把它测出来，并从真实 Trace 中寻找新的机制边界。

- Goel et al., “Great Models Think Alike and this Undermines AI Oversight”, ICML 2025: 研究模型相似性、错误重叠与 AI oversight。
- Verma et al., “AgentJudgeBench”, arXiv:2608.26623, 2026: 专门评估 agentic tool-calling 场景的 LLM judge 可靠性。
- Dev et al., “Judge Reliability Harness”, arXiv:2603.05399, 2026: 对 judge 在扰动、格式变化等条件下做可靠性压力测试。
- Qiao et al., “VerifyMAS”, arXiv:2605.17467, 2026: 用 hypothesis verification 做多智能体失败归因。
- τ -bench / tau2-bench v1.0.1: 本课程继续使用其 text half-duplex、tool type 与 trajectory 结构。

12. 常见误区

误区	为什么错	正确做法
Verifier accuracy 高，所以系统一定更可靠	可能只在简单样本上高，关键错误上共盲	看 dangerous co-failure 与条件错误率
同一个强模型做 Agent + Verifier 最稳	能力强不等于错误互补	比较 same-model 与 cross-model
reference action 不匹配就是 Agent 错	reference trajectory 通常非唯一	人工结合 policy + state 标注
$\kappa > 1$ 就能发论文	相关信号不是因果机制	精读 Trace + 最小干预验证
多跑几个 episode 就够	真正单位可能是 WRITE proposal	明确统计单位和样本量

13. 本讲最低闭环能力

你学完后至少要能独立完成：① 解释为什么组件准确率不能直接推出系统可靠性；② 跑 Shadow Verifier；③ 人工标 20~40 个 WRITE proposals；④ 计算四象限、 κ 、 ϕ ；⑤ 找出 3~5 个危险双错 Trace；⑥ 写出一个可证伪的 shared-blind-spot 假设。